

GITHUB COPILOT · ENGINEERING

AI 编码如何 降本不降质

真正该优化的不是单次 token，而是完整任务

Erik Kristensen & Napalys Klicius · 2026.09.02

核心命题

输出更短，为什么反而可能更贵？



单次响应
更短

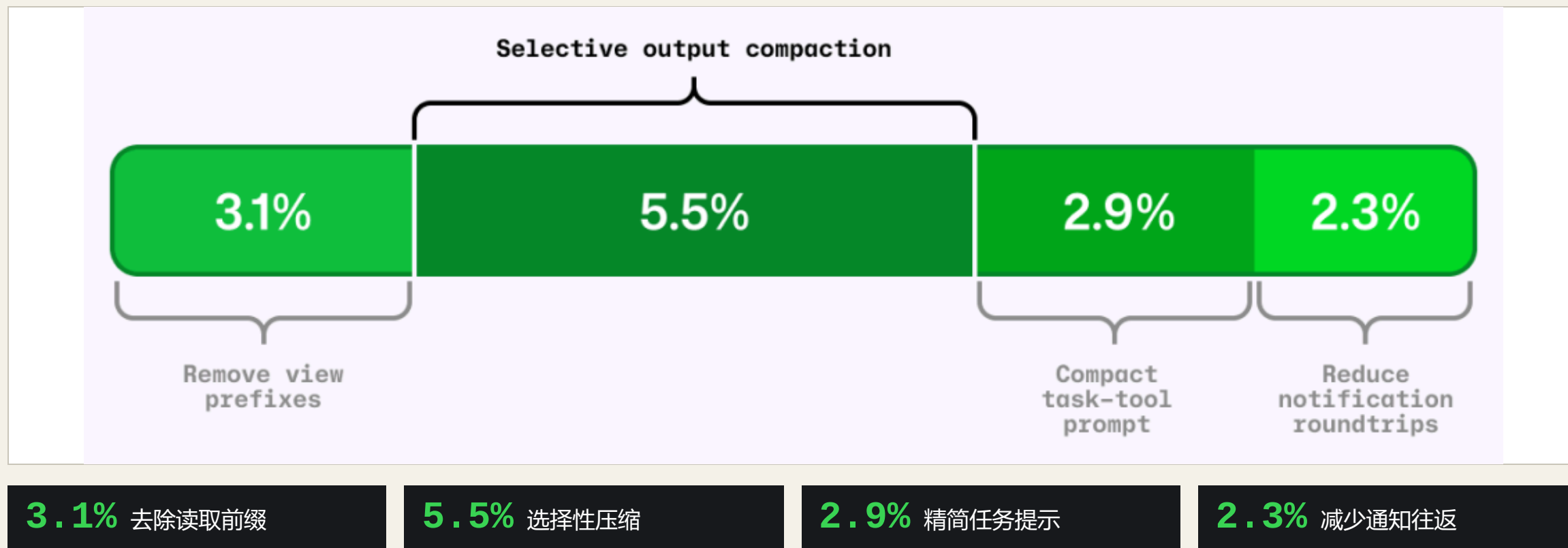
≠



完整任务
更高效

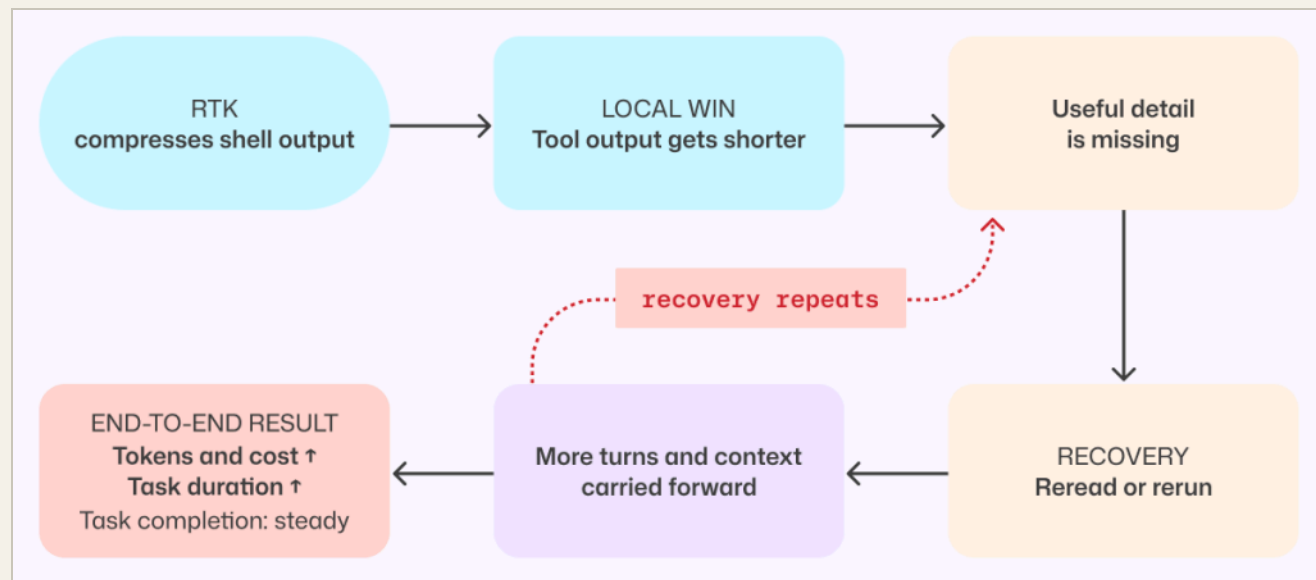
缺失的信息会迫使智能体重读、重跑、增加轮次，并把更多上下文一路带下去。

优化结果，而不是优化一次工具调用



四组 A/B 实验使用相同 AI Credits 指标，效果不一定严格相加。

省在工具输出，花在恢复路径



RTK 确实缩短了部分 shell 输出，但省略的细节有时正是下一步所需。

信息丢失 → 重读或重跑 → 成本上升

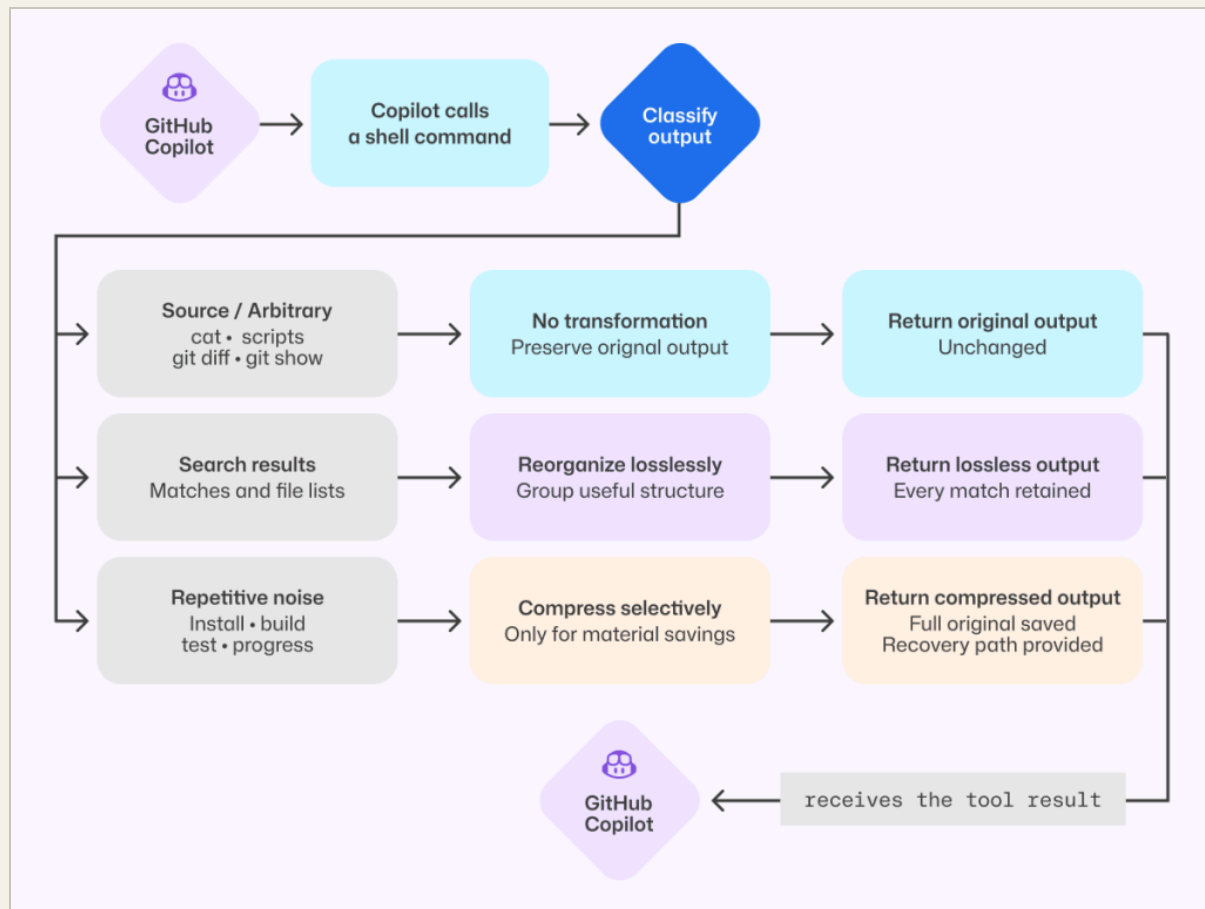
结论只适用于被测集成与负载，但足以推翻“每次调用越短越好”的目标。

FOUR CHANGES

减少无用工作 保留必要信息

压缩输出 · 删除格式 · 精简提示 · 消除往返

按输出“代表什么”决定是否压缩



01 原样保留
cat、git diff、git show 与任意脚本输出

02 无损重排
grep 等搜索结果分组，但不丢任何匹配

03 选择压缩
仅压缩 install、build、test 的重复噪声

恢复路径既是保险，也是评估信号



打开完整原始输出



重新运行命令



重复或缩窄搜索



增加模型轮次

离线：未检测到任务成功率显著下降，智能体极少打开原始输出 · **线上：**平均成本小幅下降，质量指标无实质回退

频繁恢复 = 压缩器删掉了有价值的信息

先删格式，不删信息

BEFORE • numbered output

```
1. import type { User } from "./types.js";
2.
3. export function greeting(user: User) {
4.     return 'Hello, ${user.name}'
5. }
```

AFTER • original source only

```
import type { User } from "./types.js";

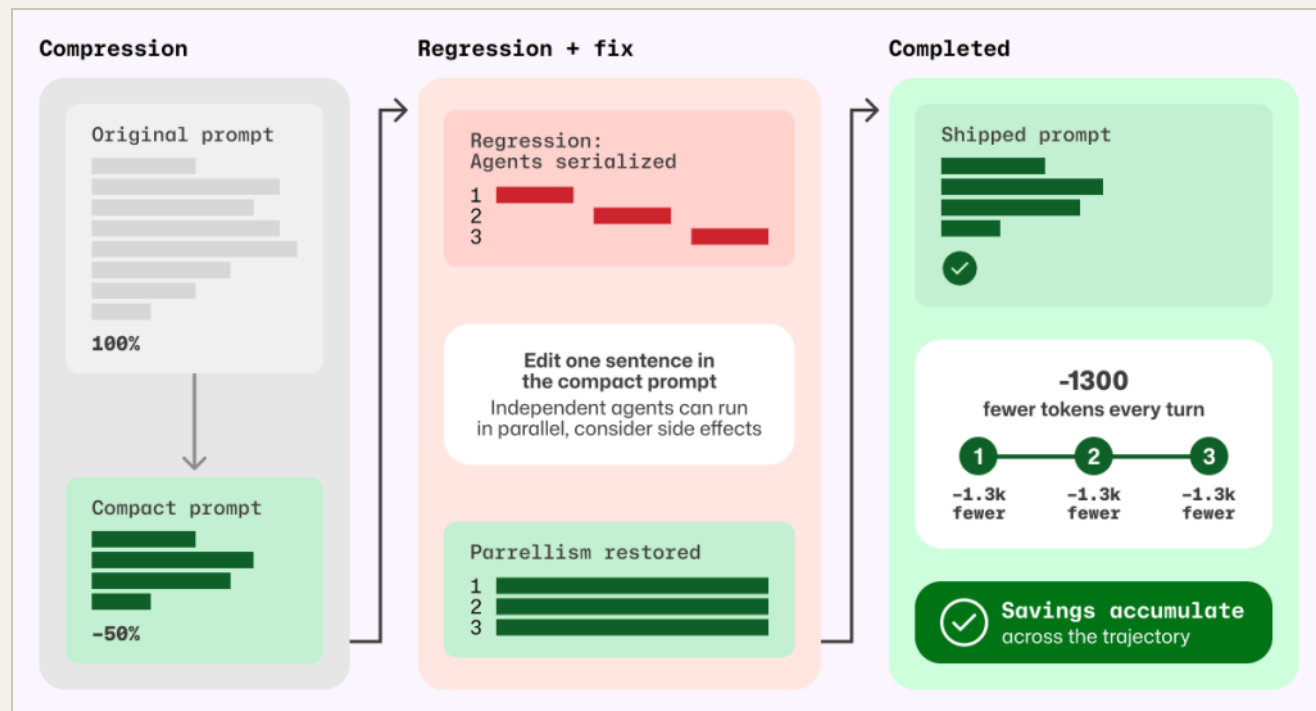
export function greeting(user: User) {
    return 'Hello, ${user.name}'
}
```

约 5% 离线基准推理成本下降

约 3% CLI 用户日均推理成本下降

行号仍适合 diff 和短代码片段，却不该附着在每次完整文件读取上。

压缩提示词，但不能压缩意图

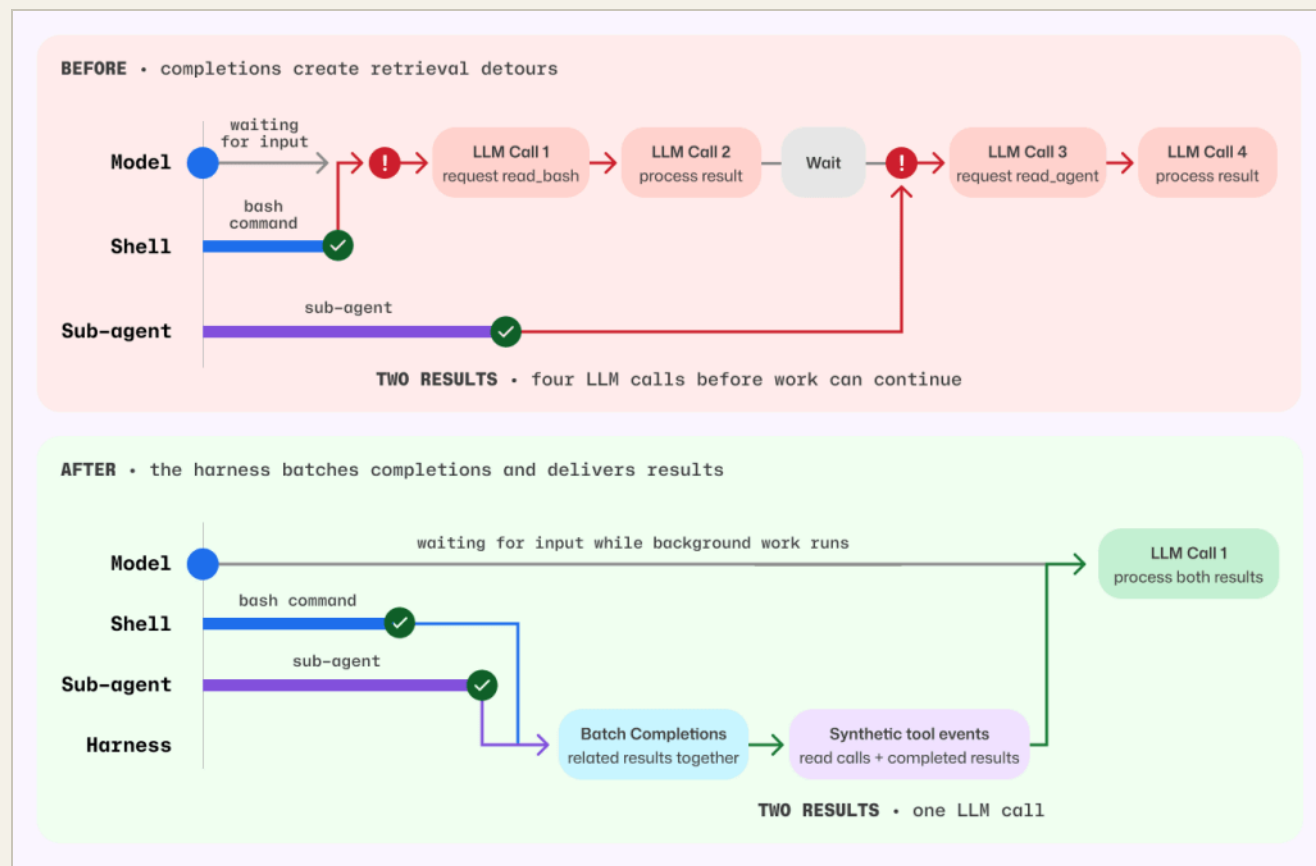


元提示循环把 task 工具提示缩短约一半，却意外把谨慎建议改成硬性调度规则。

“独立智能体可以并行运行；考虑副作用。”

新增回归测试后，这一句恢复了并行行为，同时每轮减少约 1,300 个提示 token。

任务完成时，直接把结果送回来



以前

4 次模型调用

取 shell 结果、处理结果、取子智能体结果、再处理

现在

1 次模型调用

批量通知，直接附带完整结果

不压缩、不总结、不扣留信息，只移除纯检索往返，
AI Credits 平均降低约 **2.3%**。

证据是局部的：每个 workflow 都要重测

未上线

更紧的文件工具指令

在 code review 中有效，却在 Copilot CLI 在线实验中增加成本。

已上线

去行号 + 选择性压缩

在大规模 code review 评估中，各自减少约 5% 平均提示 token，质量无实质变化。

同一项优化，换一个产品表面，就可能得出相反结论。

构建高效 AI 编码智能体的原则

1 优化完整任务
不是单次工具调用

2 优化编排
消除确定性的模型往返

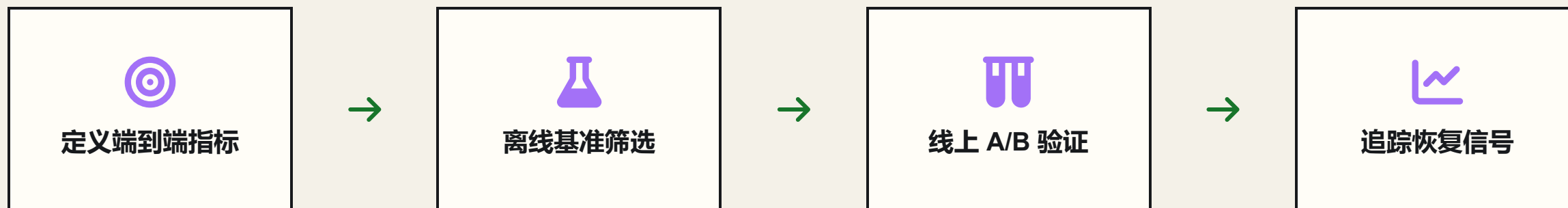
3 按语义压缩
优先无损，并保留恢复路径

4 测试提示行为
防止精简时悄悄删掉意图

5 在本地验证证据
跨基准、线上实验和产品重测

落地清单

把成本优化变成工程闭环



每次准备“省 token”之前，先问：**这会不会让智能体之后重新找回来？**

TAKEAWAY

模型没有变聪明 只是少做了无用功

高效，不是让上下文更少，而是让每一份上下文都推动任务向前。

完整任务

保留信息

减少往返

持续验证